

UT9 - DW1A – BASES DE DATOS

Bases de Datos Objeto-Relacionales

Contenido

Introducción a las Bases de Datos Objeto-Relacionales.....	2
Características de las Bases de Datos Objeto-Relacionales	2
Tipos de datos definidos por el usuario (UDT)	2
Encapsulación	2
Herencia.....	2
Colecciones.....	2
Tablas de objetos	3
Columnas tipo objeto	3
Métodos asociados	3
Creación de Tipos de Datos Objeto	3
Ejemplo: tipo DIRECCION	3
Ejemplo con método.....	3
Tablas de Objetos y Tablas con Columnas Tipo Objeto.....	4
Tablas de objetos.....	4
Tablas con columnas tipo objeto	4
Tipos Colección	4
VARRAY	4
NESTED TABLE.....	4
Uso dentro de un objeto	4
Consultas sobre Objetos y Colecciones.....	4
Seleccionar atributos de un objeto	4
Consultar tablas de objetos	5
Consultar colecciones (NESTED TABLE)	5
Modificación de Objetos y Colecciones.....	5
Insertar objetos	5
Actualizar atributos.....	5
Insertar colecciones	5
Mantener integridad y consistencia.....	5
Eliminación en orden correcto (DROP)	6

Introducción a las Bases de Datos Objeto-Relacionales

Las bases de datos objeto-relacionales (BDOR) combinan:

- El **modelo relacional tradicional** (tablas, filas, columnas, claves)
- Con **conceptos del paradigma orientado a objetos** (clases, atributos, métodos, encapsulación, herencia)

Su objetivo es superar las limitaciones del modelo relacional cuando se trabaja con:

- Datos complejos
- Estructuras jerárquicas
- Objetos con comportamiento
- Colecciones
- Multimedia

Uno de los gestores más adecuado para trabajo con estas bases de datos es Oracle Database, porque:

- Implementa completamente el modelo objeto-relacional
- Permite crear tipos objeto, métodos, colecciones, tablas de objetos
- Tiene una versión gratuita: **Oracle Database Express Edition (XE)**

Características de las Bases de Datos Objeto-Relacionales

Las BDOR permiten:

Tipos de datos definidos por el usuario (UDT)

- Equivalentes a clases
- Pueden tener atributos y métodos

Encapsulación

Los datos y métodos se agrupan en un único tipo objeto.

Herencia

Un tipo puede derivar de otro.

Colecciones

- VARRAY
- NESTED TABLE

Tablas de objetos

Tablas cuyos registros son instancias de un tipo objeto.

Columnas tipo objeto

Columnas que almacenan objetos dentro de una tabla relacional.

Métodos asociados

Funciones o procedimientos que operan sobre los atributos del objeto.

Creación de Tipos de Datos Objeto

En Oracle se usa:

```
CREATE TYPE nombre_tipo AS OBJECT (  
    atributo1 tipo,  
    atributo2 tipo,  
    ...  
    MEMBER FUNCTION ...  
    MEMBER PROCEDURE ...  
);
```

Ejemplo: tipo DIRECCION

```
CREATE TYPE direccion_t AS OBJECT (  
    calle VARCHAR2(50),  
    ciudad VARCHAR2(30),  
    cp NUMBER(5)  
);
```

Ejemplo con método

```
CREATE TYPE empleado_t AS OBJECT (  
    dni VARCHAR2(9),  
    nombre VARCHAR2(50),  
    salario NUMBER,  
    MEMBER FUNCTION salario_anual RETURN NUMBER  
);
```

Tablas de Objetos y Tablas con Columnas Tipo Objeto

Tablas de objetos

Cada fila es un objeto completo.

```
CREATE TABLE empleados_obj OF empleado_t;
```

Tablas con columnas tipo objeto

Tabla relacional que contiene objetos como columnas.

```
CREATE TABLE pedidos (  
    id NUMBER PRIMARY KEY,  
    fecha DATE,  
    direccion_envio direccion_t  
);
```

Tipos Colección

Oracle permite dos tipos:

VARRAY

Colección con tamaño máximo fijo.

```
CREATE TYPE telefonos_t AS VARRAY(5) OF VARCHAR2(15);
```

NESTED TABLE

Colección sin límite de tamaño.

```
CREATE TYPE aficiones_t AS TABLE OF VARCHAR2(30);
```

Uso dentro de un objeto

```
CREATE TYPE persona_t AS OBJECT (  
    nombre VARCHAR2(50),  
    telefonos telefonos_t,  
    aficiones aficiones_t  
);
```

Consultas sobre Objetos y Colecciones

Seleccionar atributos de un objeto

```
SELECT p.nombre, p.direccion_envio.ciudad  
FROM pedidos p;
```

Consultar tablas de objetos

```
SELECT e.nombre, e.salario  
FROM empleados_obj e;
```

Consultar colecciones (NESTED TABLE)

```
SELECT p.nombre, t.COLUMN_VALUE AS aficion  
FROM personas p,  
TABLE(p.aficiones) t;
```

Modificación de Objetos y Colecciones

Insertar objetos

```
INSERT INTO empleados_obj  
VALUES (empleado_t('12345678A', 'Ana', 25000));
```

Actualizar atributos

```
UPDATE empleados_obj e  
SET e.salario = e.salario + 2000  
WHERE e.dni = '12345678A';
```

Insertar colecciones

```
INSERT INTO personas  
VALUES (  
    persona_t(  
        'Luis',  
        telefonos_t('600123123', '911223344'),  
        aficiones_t('fútbol', 'cine')  
    )  
);
```

Mantener integridad y consistencia

- Validar tipos
- Usar métodos para encapsular lógica
- Evitar colecciones inconsistentes
- Usar transacciones (COMMIT, ROLLBACK)

Eliminación en orden correcto (DROP)

- **Las tablas** deben borrarse primero porque dependen de los tipos.

```
DROP TABLE personas;
```

```
/
```

- **El cuerpo** del método debe eliminarse antes del tipo.

```
DROP TYPE BODY empleado_t;
```

```
/
```

- Los **tipos objeto** se eliminan al final porque son la base de todo el modelo.

```
DROP TYPE persona_t;
```

```
/
```

- **Tipos colección** no pueden borrarse si aún existen tipos que los usan.

```
DROP TYPE aficiones_t;
```

```
/
```